ORACLE TRUTH FORGE - INTERNAL DOC

# Oracle - Le produit cote membre, explique a 360

**Version** · 21 avril 2026

**Branche** · crm-integration

**Public** · Charles (founder), equipe interne

## Sommaire

Oracle — Le produit côté membre, expliqué à 360°

Sommaire

1. Oracle, c'est quoi exactement ?
2. Les 4 types de personnes qui utilisent Oracle
   - 🎓 Le Membre (paid customer)
   - ⏱️ L'Early Access (trial)
   - 👨‍💼 L'Admin (équipe interne)
   - 📞 Le Setter (équipe commerciale)
3. Le parcours d'un nouveau venu (de zéro à actif)
   - Étape 1 — Il voit une pub (jour J)
   - Étape 2 — Il atterrit sur la Landing Page (jour J)
   - Étape 3 — Il clique → page Apply (jour J)
   - Étape 4 — Il atterrit sur Discovery (jour J)
   - Étape 5 — Il atterrit sur Final (jour J)
   - Étape 6 — Le call avec le closer (jour J+1 à J+7)
   - Étape 7 — Le prospect reçoit le magic link
   - Étape 8 — Premier login dans le dashboard
   - Étape 9 — Il commence son trial (jours J+1 à J+8)
   - Étape 10 — Le timer EA expire
   - Étape 11 — Il paye (devient membre)
4. Les pages que voit un membre (et ce qu'il y fait)
   - 🎯 Exécution d'Oracle (tab=execution)
   - 🗄️ Setup (tab=setup)
   - 📊 Data Analysis (tab=data-analysis)
   - 🎬 Vidéo du Setup Oracle (tab=videos)
   - 🏆 Chat / Successes (tab=successes)
   - 🥇 Résultats (tab=results, EA seulement)
5. Les 4 concepts magiques d'Oracle
   - 🌀 Concept 1 : Les Cycles
   - 📝 Concept 2 : Les 3 types de trades
   - 🎮 Concept 3 : Les Quêtes
   - ✅ Concept 4 : Les Verifications
6. Comment ça marche derrière (la base de données)
   - Les tables principales (les "stars" du modèle)
     - Côté identité du user

Côté produit
Côté contenu
Côté gamification
Côté CRM / acquisition
Côté admin / EA settings

Comment ça communique : RPC, RLS, Edge Functions

Les RPC (Remote Procedure Calls) — fonctions SQL appelables depuis le frontend
RLS (Row Level Security) — qui peut lire/écrire quoi
Les Edge Functions — code serveur custom
Storage (images)

Les triggers automatiques

7. Les règles invisibles (RLS, RPC, sécurité)
   1. Le statut du user gère son accès
   2. Le timer EA est intransigeant
   3. Multi-device : 5 max
   4. Le magic link
   5. Realtime partout
8. Les zones grises à clarifier
   🚧 1. La page Setup vs Sessions
   🚧 2. La gestion des users (3 endroits)
   🚧 3. Le bloc "Overview Globale" retiré
   🚧 4. Le bloc "Phase d'ébauche" retiré
   🚧 5. Le tab "Setup" sera renommé ?
   🚧 6. Routing du bouton "Récolter ma data"
   🚧 7. Cal.com webhook
   🚧 8. Migration vers main

Conclusion

---

## 1. Oracle, c'est quoi exactement ?

Oracle est une **plateforme web** qui aide des traders à **devenir bons**.

Au lieu de leur donner une stratégie clé en main, Oracle leur donne :

- **Une bibliothèque de 314 trades passés** (la "data Oracle"), chacun étant un exemple parfait avec setup, timing, RR, screenshot
- **Une méthode d'apprentissage en cycles** : tu études les exemples, puis tu reproduis, puis tu trades en live
- **Un journal de trading personnel** : tu loggues tes trades, tu les analyses

- **Une équipe de vérification** : des admins valident ton travail à chaque étape
- **Une communauté** : tu partages tes wins, tu vois ceux des autres

L'idée centrale : **AVE = Apprentissage → Vérification → Exécution**.

C'est un produit de formation premium pour traders. Le membre paye pour avoir accès à la data Oracle + à l'accompagnement.

---

## 2. Les 4 types de personnes qui utilisent Oracle

Imagine Oracle comme un **club privé**. Il y a 4 catégories de personnes :

### 🎓 Le Membre (paid customer)

- Il a payé son abonnement
- Il a accès à tout : la bibliothèque Oracle, son journal perso, les vidéos, les vérifications, la communauté
- C'est le client final, celui pour qui le produit existe

### ⏱️ L'Early Access (trial)

- C'est un futur membre qui teste pendant 7 jours (modulable, jusqu'à 14 jours)
- Il voit une partie du produit (les bases)
- Un timer compte à rebours dans son interface
- Au bout du timer, son accès se coupe → il doit payer pour continuer

### 👨‍💼 L'Admin (équipe interne)

- C'est Charles ou un membre de son équipe
- Il valide les cycles des membres, gère le contenu, surveille les abus
- Il voit tout ce que voit un membre, plus des outils admin

### 📞 Le Setter (équipe commerciale)

- C'est un closer/setter qui fait du sales
- Il voit uniquement le **CRM** (les leads à contacter, les calls à faire)
- Il ne voit pas le produit en lui-même

Et il y a deux **étiquettes** spéciales : - 🏢 **Institute** : un client institutionnel (ne voit pas le mur des wins de la communauté, c'est confidentiel pour lui) - 💎 **is_client** : flag interne, marque que ce membre a payé

Aujourd'hui ces 4 types se gèrent dans 3 endroits différents (CRM, Gestion, Config). Pas idéal — voir les pending decisions.

---

## 3. Le parcours d'un nouveau venu (de zéro à actif)

Voici ce qui se passe **chronologiquement** quand quelqu'un découvre Oracle :

### Étape 1 — Il voit une pub (jour J)

Le futur prospect tombe sur une pub Oracle (Facebook, Insta, YouTube). Il clique.

### Étape 2 — Il atterrit sur la Landing Page (jour J)

URL : `/vip/landing` Il voit un titre punchy + un CTA "Commencer". Pas de form, juste de l'envie.

### Étape 3 — Il clique → page Apply (jour J)

URL : `/vip/apply` - En haut : une **VSL** (vidéo de vente Vidalytics) avec un titre accrocheur ("Si je te donne accès gratuitement à un système qui te permet de vérifier le potentiel de chaque trade...") - Il regarde la vidéo - Il clique sur "Candidater" → un **formulaire** s'ouvre - 8 questions (objectif, temps disponible, expérience, budget, etc.) - Si une de ses réponses est **disqualifiante** (ex : budget < 1000€), le form lui dit gentiment "merci pour ton honnêteté" et s'arrête là - Sinon, il rentre nom + tél + email - Submit → ses infos vont dans la table `early_access_requests` avec status `en_attente`

### Étape 4 — Il atterrit sur Discovery (jour J)

URL : `/vip/discovery` - Un calendrier **Cal.com** est embedded - Il choisit un créneau pour un call avec un closer - Il valide la réservation - Cal.com envoie un **webhook** à Oracle → l'entrée dans `early_access_requests` est mise à jour : `call_booked = true`, `call_scheduled_at = ...`

### Étape 5 — Il atterrit sur Final (jour J)

URL : `/vip/final?date=...&email=...` - Page de confirmation : "Bravo, ton appel est réservé" - 3 instructions : ajoute au calendrier, vérifie tes spams, sois à l'heure - Champ optionnel : "Une question avant l'appel ?" → si remplie, ça update `early_access_requests.precall_question`

### Étape 6 — Le call avec le closer (jour J+1 à J+7)

Hors plateforme. Le closer parle au prospect. Si ça matche, il **approuve** le lead dans le CRM Oracle : - L'admin/closer clique "Approuver" dans le CRM - L'edge function `approve-early-access` se déclenche - Elle crée un user dans `auth.users` - Elle crée un row dans `profiles` avec `status = 'pending'` (le user n'a pas encore set son mdp) - Elle assigne le rôle `early_access` dans `user_roles` avec `expires_at = NULL` - Elle envoie un **magic link** par email au prospect

### Étape 7 — Le prospect reçoit le magic link

- Il clique le lien dans son email
- Il atterrit sur `/setup-password` → il choisit son mot de passe
- `profiles.status` passe à `active`

- Il est redirigé sur `/dashboard`

### Étape 8 — Premier login dans le dashboard

- Au premier login, le RPC `activate_ea_timer()` est appelé
- Il pose `expires_at = NOW() + 7 jours` dans `user_roles` (ou la durée configurée)
- Le timer commence
- Le RPC `initialize_user_cycles()` crée 9 entrées dans `user_cycles` (ébauche + cycles 1-8) avec status `locked`, sauf l'ébauche qui devient `in_progress`

### Étape 9 — Il commence son trial (jours J+1 à J+8)

- Il navigue dans le dashboard
- Il regarde les vidéos de formation
- Il analyse les 15 trades de l'ébauche (case à cocher)
- Quand il a fini : il soumet une demande de vérification
- L'admin valide ou rejette

### Étape 10 — Le timer EA expire

- Au bout de 7 jours, `expires_at < NOW()`
- La fonction RLS `is_early_access()` retourne `false`
- Le user voit toujours son dashboard mais sans accès aux données Oracle (RLS bloque)
- Une popup l'invite à upgrader (book un call de close)

### Étape 11 — Il paye (devient membre)

- L'admin lui change son rôle de `early_access` → `member`
- Profile : `is_client = true`
- Plus de timer, accès permanent
- Il peut continuer ses cycles 1-8 jusqu'à diplomation

---

## 4. Les pages que voit un membre (et ce qu'il y fait)

Une fois connecté au dashboard, le membre voit une **sidebar à gauche** avec ses pages. Voici chacune :

### 🎯 Exécution d'Oracle ( `tab=execution` )

**Fichier** : `src/components/dashboard/OracleExecution.tsx`

C'est sa **page d'accueil**. Il y voit : - Un titre "Exécution d'Oracle" + le sous-titre AVE - 3 cartes : Apprentissage (vidéos), Vérification (récolter data), Exécution (coming soon) - Pour les EA : un **trade featured** sélectionné par l'admin (un trade du jour à analyser) - Pour les EA : ses résultats personnels embed - Une bulle de **quêtes** flottante : où en est-il ?

**À quoi ça sert** : c'est le hub. Il décide ici ce qu'il fait : regarder une vidéo, ouvrir son setup, voir ses résultats.

### 🗄️ Setup ( `tab=setup` )

**Fichier** : `src/components/dashboard/SetupPage.tsx`

C'est sa **bibliothèque de trades**. 3 sous-vues :

1. **Overview** : stats globales sur les 314 trades Oracle (combien de Long, RR moyen, par jour de la semaine, etc.)
2. **Oracle Database** : la liste des 314 trades avec filtres (direction, setup, screenshot, RR, etc.). Cliquable pour voir le détail + screenshots.
3. **Setup Perso** : SES trades à lui qu'il a loggés à la main (différents des 314 Oracle).

**À quoi ça sert** : étudier les exemples Oracle + tenir son journal perso.

### 📊 Data Analysis ( `tab=data-analysis` )

**Fichier** : `src/components/dashboard/DataAnalysisPage.tsx`

Statistiques sur les trades : - Pour un membre normal : **uniquement ses trades persos** - Pour un EA : ses persos + les trades Oracle (compare) - Pour un admin : tout le monde

Stats : win rate, RR moyen, par setup, par direction, heatmap par jour, etc.

**À quoi ça sert** : voir où il performe, où il échoue, identifier ses biais.

### 🎬 Vidéo du Setup Oracle ( `tab=videos` )

**Fichier** : `src/components/dashboard/VideoSetup.tsx`

Liste de vidéos de formation. Chaque vidéo est embed (YouTube/Vimeo). Quand il clique → marquée comme vue dans `user_video_views`.

Pour les EA : il y a aussi des **bonus videos**.

**À quoi ça sert** : la formation théorique. Sans ça pas d'ébauche complète.

### 🏆 Chat / Successes ( `tab=successes` )

**Fichier** : `src/components/dashboard/SuccessPage.tsx`

C'est le **mur des wins** de la communauté : - Tout le monde poste ses screenshots de payouts, validations de prop firms, etc. - Il peut poster les siens - Il y a un leaderboard - Indicateur "qui est en ligne" (heartbeat via `ea_activity_tracking`)

**Caché pour les institute** (clients institutionnels qui ne veulent pas voir/être vus).

**À quoi ça sert** : motivation, social proof, communauté.

### 🥇 Résultats ( `tab=results` , EA seulement)

**Fichier** : `src/components/dashboard/ResultsPage.tsx`

Galerie de **résultats officiels Oracle** : payouts de la company, validations de challenges. Curated par l'admin.

**À quoi ça sert** : prouver à l'EA pendant son trial que la méthode marche (proof of concept).

---

## 5. Les 4 concepts magiques d'Oracle

### 🌀 Concept 1 : Les Cycles

Le voyage d'un membre est découpé en **9 cycles** :

| # | Nom | Range trades | Phase | Total trades |
|---|---|---|---|---|
| 0 | Ébauche | 1-15 | Onboarding | 15 |
| 1 | Cycle 1 | 16-40 | Phase 1 | 25 |
| 2 | Cycle 2 | 41-65 | Phase 1 | 25 |
| 3 | Cycle 3 | 66-90 | Phase 1 | 25 |
| 4 | Cycle 4 | 91-115 | Phase 1 | 25 |
| 5 | Cycle 5 | 116-165 | Phase 2 | 50 |
| 6 | Cycle 6 | 166-215 | Phase 2 | 50 |
| 7 | Cycle 7 | 216-265 | Phase 2 | 50 |
| 8 | Cycle 8 | 266-314 | Phase 2 | 50 |

**Comment ça marche** : - Au départ, **seul l'ébauche est** `in_progress` , les autres sont `locked` - Pour l'ébauche : il analyse 15 trades (case à cocher) + regarde les vidéos - Quand il a fini, il **soumet une demande de vérification** → row dans `verification_requests` - L'admin valide ou rejette - Si validé : ébauche `validated` , cycle 1 unlocked en `in_progress` - Pour les cycles 1-8 : il **logge ses propres exécutions** (ses trades reproduits) dans `user_executions` - Quand il atteint le total (25 ou 50), il soumet pour vérification - L'admin valide → cycle suivant unlocked

C'est un parcours **séquentiel et obligatoire**. Pas de raccourcis.

### 📝 Concept 2 : Les 3 types de trades

C'est subtil mais important :

| Table | Qui possède | Description |
|---|---|---|
| `trades` | Oracle (master) | Les 314 trades de référence créés par l'équipe Oracle |
| `user_executions` | Membre | Pour chaque trade Oracle, le membre logge **sa version** (ce qu'il aurait fait) |
| `user_personal_trades` | Membre | Trades **complètement perso** que le membre fait en dehors d'Oracle |

**Exemple concret** : - Le trade Oracle #42 est un Long sur NASDAQ le 12 mars 2025 à 14:30 avec RR 2.5 - Le membre, dans son cycle 2, doit reproduire ce trade. Il logge dans `user_executions` : "Trade #42, j'aurais entré à 14:35 (5 min de retard), RR final 2.0" - En parallèle, le même membre fait du day trading sur EUR/USD ce week-end. Il logge ce trade dans `user_personal_trades` (pas lié à Oracle).

### 🎮 Concept 3 : Les Quêtes

Système de **gamification** pour motiver le membre. Quêtes typiques : - "Logge 5 trades gagnants aujourd'hui" (daily) - "Analyse les 15 trades de l'ébauche" - "Regarde toutes les vidéos" - "Connecte FX Replay"

Tracké dans `user_quest_flags` + différentes tables ( `user_executions` , `user_video_views` , etc.).

Affiché dans une **bulle flottante** + carte "Daily Quest" sur la home.

### ✅ Concept 4 : Les Verifications

Pour passer d'un cycle au suivant, il faut **être validé par un admin humain**.

Workflow : 1. Membre clique "Demander une vérification" sur son cycle 2. Row dans `verification_requests` avec status `pending` 3. Admin va dans **Vérif. Admin** ou **Gestion** 4. Il review les trades du membre, compare avec la master Oracle 5. Il valide ( `approved` ) ou rejette ( `rejected` ) avec un commentaire ( `admin_comments` ) 6. Si validé : `user_cycles[ce_cycle].status = 'validated'` , cycle suivant unlock 7. Si rejeté : `user_cycles[ce_cycle].status = 'rejected'` , le membre doit refaire des trades + soumettre à nouveau

C'est l'humain dans la boucle qui garantit la qualité.

---

## 6. Comment ça marche derrière (la base de données)

Toute la base de données est sur **Supabase** (PostgreSQL hébergé). Il y a une **30aine de tables**.

## Les tables principales (les "stars" du modèle)

### Côté identité du user

| Table | Rôle |
|---|---|
| `auth.users` | Géré par Supabase. Stocke email, password hash, metadata. |
| `profiles` | Notre table métier. Lie un `user_id` à un nom, statut, infos perso. |
| `user_roles` | Plusieurs rôles possibles par user. Stocke aussi `expires_at` pour les EA. |
| `user_sessions` | Les sessions actives par device. Anti-piratage : max 5 devices. |

### Côté produit

| Table | Rôle |
|---|---|
| `cycles` | Les 9 cycles (référence statique). |
| `user_cycles` | Pour chaque user × chaque cycle, le statut + progression. |
| `trades` | Les 314 trades master Oracle. |
| `user_executions` | Pour chaque user × chaque trade, son log d'exécution. |
| `user_personal_trades` | Trades persos hors Oracle. |
| `user_trade_analyses` | Pour l'ébauche : quels trades 1-15 le user a "analysé" (case cochée). |
| `verification_requests` | Demandes de validation cycle par cycle. |
| `admin_trade_notes` | Notes de l'admin pendant la verif (par trade). |

### Côté contenu

| Table | Rôle |
|---|---|
| `videos` | Vidéos de formation principales. |
| `bonus_videos` | Vidéos bonus (EA + admin only). |
| `user_video_views` | Qui a vu quoi. |
| `results` | Galerie de résultats Oracle (preuves). |
| `user_successes` | Wins postés par les membres dans le mur. |

### Côté gamification

| Table | Rôle |
|---|---|
| `quest_step_configs` | Définitions des étapes de quête. |
| `user_quest_flags` | Flags custom (ex: fxreplay_connected). |

### Côté CRM / acquisition

| Table | Rôle |
|---|---|
| `early_access_requests` | Lead capturé via le funnel. Une row par prospect. |
| `lead_events` | Timeline d'événements par lead (form_submitted, contacted, call_booked, etc.). |
| `lead_comments` | Notes d'équipe sur un lead. |
| `funnels` + `funnel_config` | Configuration des pages funnel (landing, apply, discovery, final). |

### Côté admin / EA settings

| Table | Rôle |
|---|---|
| `ea_activity_tracking` | Heartbeat des EA (quel tab, dernière activité). |
| `ea_lead_notes` | Notes legacy sur les leads EA. |
| `ea_featured_trades` | Trade du jour mis en avant pour les EA. |
| `ea_settings` | Config UX EA (URLs des boutons, etc.). |
| `security_alerts` | Alertes de sécurité (3e device, etc.). |

## Comment ça communique : RPC, RLS, Edge Functions

### Les RPC (Remote Procedure Calls) — fonctions SQL appelables depuis le frontend

| RPC | Rôle |
|---|---|
| `is_admin()` | Retourne true si l'user courant est admin. |
| `is_super_admin()` | Pareil pour super_admin. |
| `is_setter()` | Pareil pour setter. |
| `is_early_access()` | True si EA actif (timer non expiré). **Cette fonction a été corrigée le 12 avril** pour vérifier `expires_at`, sinon les EA expirés gardaient l'accès. |
| `is_institute()` | True si rôle institute. |
| `has_role(role)` | Check générique. |
| `activate_ea_timer()` | Active le timer EA au premier login. |
| `initialize_user_cycles(user_id)` | Crée les 9 entrées user_cycles à l'inscription. |
| `check_cycle_accuracy_and_auto_validate()` | (admin) Auto-valide un cycle si l'accuracy est suffisante. |

### RLS (Row Level Security) — qui peut lire/écrire quoi

Chaque table a des **policies** qui définissent les permissions par rôle. Exemples :

- `user_executions` : un user ne peut voir que SES propres exécutions. Un admin voit tout.
- `trades` : tout le monde authentifié peut voir, mais EA expirés non (RLS check `is_early_access()`).
- `user_roles` : seul un super_admin peut INSERT/UPDATE.

C'est la **sécurité au niveau base de données** — même si quelqu'un bypass le frontend, la DB refuse les requêtes non autorisées.

### Les Edge Functions — code serveur custom

| Function | Rôle |
|---|---|
| `approve-early-access` | Quand l'admin approuve un lead, cette fonction crée le compte auth + assigne le rôle EA + envoie magic link. |
| `cal-webhook` | Reçoit les webhooks de Cal.com quand un lead réserve/annule un call. Met à jour `early_access_requests` + émet des events dans `lead_events`. |

### Storage (images)

4 buckets Supabase : - `avatars` (public) : photos de profil - `trade-screenshots` (privé, signed URLs) : screenshots des trades - `success-screenshots` (privé) : screenshots des wins - `result-screenshots` (privé) : preuves de résultats Oracle

### Les triggers automatiques

À l'inscription ( `auth.users INSERT` ) : - Trigger crée automatiquement un row dans `profiles` - Trigger assigne automatiquement le rôle `member` dans `user_roles`

---

## 7. Les règles invisibles (RLS, RPC, sécurité)

Quelques règles importantes pour comprendre **ce qui se passe sans que l'utilisateur le voie** :

### 1. Le statut du user gère son accès

- `pending` : ne peut pas se connecter
- `active` : peut se connecter, accès selon rôle
- `frozen` : déconnecté de force, doit contacter admin
- `banned` : bloqué définitivement

### 2. Le timer EA est intransigeant

- Tant que `expires_at > NOW()` , l'EA voit tout
- Dès que `expires_at < NOW()` , RLS bloque l'accès aux trades Oracle
- Le user voit son dashboard mais avec contenu vide + popup pour upgrade

### 3. Multi-device : 5 max

- Chaque login enregistre un **device fingerprint** ( `platform | resolution | language` )
- Si l'user dépasse 5 devices distincts → compte gelé automatiquement + alerte admin

### 4. Le magic link

- Quand un lead est approuvé, Supabase Auth envoie un magic link via SMTP Lovable Cloud
- Le user clique → atterrit sur `/setup-password`
- Il doit set un mdp avant d'accéder au dashboard

### 5. Realtime partout

- Les pages s'auto-rafraîchissent quand la DB change (via `supabase.channel().on('postgres_changes', ...)` )
- Exemples : un admin valide un cycle → le membre voit le changement immédiatement
- Limites : sur localhost en dev, ça abort souvent (Vite HMR), mais en prod ça marche

## 8. Les zones grises à clarifier

Voici ce qui n'est **pas encore parfait** ou **à trancher** :

### 🚧 1. La page Setup vs Sessions

Aujourd'hui, le membre a UN setup perso (la table `user_personal_trades`). Le projet est de la transformer en **système de sessions** (backtesting + live trading) avec : - Plusieurs sessions nommées (NAS100 Breakout, EUR/USD London, etc.) - Chaque session = un journal isolé - Migration nécessaire : créer une session "default" par user existant

**Questions ouvertes** : - Sessions vs cycles Oracle = orthogonaux ou combinés ? - Limite de sessions par user ? - Qu'est-ce que "Prime Setup Oracle" ?

(Voir `oracle_pending_decisions.md`)

### 🚧 2. La gestion des users (3 endroits)

Actuellement, gérer un user passe par 3 panels : - **CRM** : pré-product (sales) - **Gestion** : post-first-login (product) - **Config > Rôles** : tous les users (admin/EA/clients)

C'est redondant. À unifier ou clarifier.

Et la décision récente : **Gestion sera visible par tout le monde** (EA, membres, équipes), donc certaines stats globales pourraient y vivre pour tous.

### 🚧 3. Le bloc "Overview Globale" retiré

Le bloc 4 stats (Data totale récoltée, Membres actifs, RR Total, RR Moyen) a été retiré de la page Exécution d'Oracle. À recréer ailleurs (Gestion ? Nouveau panel métriques ?).

**Issues techniques à corriger** avant réintégration : - Double-comptage du totalRR (additionne user_executions + trades, ce qui gonfle) - Titre "Setup Indices US" hardcodé alors que les stats couvrent tout - Définition floue de "Membres actifs"

### 🚧 4. Le bloc "Phase d'ébauche" retiré

Le `CycleProgressBar` qui affichait la progression actuelle (ébauche ou cycle courant) a été retiré. Question : où le replacer ? Dashboard dédié ? Widget dans la fiche utilisateur ?

### 🚧 5. Le tab "Setup" sera renommé ?

Plan : renommer "Setup" en "Récolte de données". Pas tranché si c'est un simple renommage ou un nouveau tab distinct (et l'ancien Setup disparaît).

### 🚧 6. Routing du bouton "Récolter ma data"

Sur la page Exécution d'Oracle, le bouton "Récolter ma data" navigue actuellement vers le tab `setup`. À voir si ça doit rester ou pointer ailleurs après le renommage.

### 🚧 7. Cal.com webhook

Le webhook `cal-webhook` est codé mais le secret `CAL_WEBHOOK_SECRET` n'est pas configuré dans Lovable Cloud → le webhook renvoie 503 si appelé. À setup avant la prod.

### 🚧 8. Migration vers main

La branche `crm-integration` a 30+ commits d'avance sur `main`. Plusieurs migrations SQL sont déjà appliquées sur le test DB mais pas sur la prod Lovable. À orchestrer proprement.

---

## Conclusion

Oracle est un produit **bien plus profond qu'il n'y paraît** : - Côté membre : un parcours en 9 cycles avec validations humaines - Côté backend : 30+ tables, RLS strict, RPCs, edge functions, realtime - Côté business : un funnel d'acquisition complet (landing → apply → discovery → final → trial → paid)

Les fondations sont solides. Les zones grises sont identifiées et trackées dans `oracle_pending_decisions.md`.

**À retenir** : avant de toucher à quoi que ce soit, se poser la question **"qu'est-ce qui existe déjà en prod et comment je migre ?"**. C'est inscrit dans le CLAUDE.md.

---

*Document généré le 21 avril 2026 — Charles + Claude*